資料結構與攤銷分析：完整詳解

A04-E01 搬移只能在 T 空時進行

操作	S（底→頂）	T（底→頂）	回傳	c	Φ	$\hat{c}$
ENQ a	a	空	—	1	2	3
ENQ b	a,b	空	—	1	4	3
DEQ	空	b	a	5	0	1
ENQ c	c	b	—	1	2	3
DEQ	c	空	b	1	2	1
DEQ	空	空	c	3	0	1

實際與攤銷成本皆為 12。第 3 步搬兩筆，成本 $2 \cdot 2 + 1 = 5$；第 6 步搬一筆，成本 3。第 5 步若把 c 也 push 到 T，它會蓋在 b 上方，錯誤地先回傳 c；應保持 b 先於 c。

A04-E02 勢能前面的係數要多大

ENQUEUE：$c = 1$、$\Delta\Phi = a$，故 $\hat{c} = 1 + a$。普通 DEQUEUE 不改 $|S|$，費用為 1。搬移 k 筆後出列：

$$\hat{c} = (2k + 1) - ak = 1 + (2 - a)k.$$

若 $a < 2$，最後一式隨 k 無界增長，不能給單次常數攤銷上界。若固定 $a \geq 2$，該式至多 1，ENQUEUE 也有固定費用 $1 + a$，所以足夠且必要。$a > 2$ 時，大搬移可能得到負費用；因勢能非負且空初態為 0，總帳仍有效。取 $a = 2$ 可讓搬移後出列剛好收 1。

A04-E03 非空初態不能免費

四筆各 pop 一次、push 一次，最後再 pop 一次，所以 $c = 2 \cdot 4 + 1 = 9$。$\Phi_0 = 8$、$\Phi_1 = 0$，故 $\Delta\Phi = -8$、$\hat{c} = 1$。

$$c = \hat{c} + \Phi_0 - \Phi_1 = 1 + 8 - 0 = 9.$$

初態含 8 單位的搬移信用，不能省略。若從空結構連同前面四次 ENQUEUE 一起計，則實際總成本 $4 + 9 = 13$，攤銷總成本 $4 \cdot 3 + 1 = 13$；完整序列的起終勢能才都為 0。

A04-E04 擴張與縮表的逐步總帳

操作	(n, M)	搬移	c	Φ	$\hat{c}$
加 a	(1,1)	0	1	1	2
加 b	(2,2)	1	2	2	3
加 c	(3,4)	2	3	2	3
加 d	(4,4)	0	1	4	3
加 e	(5,8)	4	5	2	3
刪 e	(4,8)	0	1	0	-1
刪 d	(3,8)	0	1	1	2
刪 c	(2,4)	2	3	0	2
刪 b	(1,2)	1	2	0	2
刪 a	(0,1)	0	1	0	1

搬移合計 10，成功操作 10 次，所以實際成本 20；攤銷成本也為 20。刪 e 的負費用只表示釋放兩單位信用而工作一單位，並非負執行時間。最後一次容量 2→1 不搬資料，因刪後已空。

A04-E05 半滿就縮表的反例

最初八次插入成本為 $8 + (1 + 2 + 4) = 15$，到達 $(8, 8)$。每對操作為

$$(8,8) \xrightarrow[\text{搬 } 8 \text{ 筆}]{\text{APPEND, } c=9} (9,16) \xrightarrow[\text{搬 } 8 \text{ 筆}]{\text{DELETE, } c=9} (8,8).$$

四對另花 72，總成本 87，總操作數 $m = 8 + 8 = 16$；$87 > 3m = 48$。這否定把正文的常數界直接搬到另一縮表策略，但單一數例還不是漸近反證。

一般取 2 的冪 k，建滿表需 $O(k)$，再做 k 對操作，每對成本 $2k + 2$。總共 $3k$ 次操作、成本至少 $2k^2 + 2k$，因此不存在對此操作族成立的固定常數攤銷上界。

A04-E06 勢能加常數是否破壞證明

此說法錯誤。$\Delta\Psi_i = (\Phi_i + 7) - (\Phi_{i-1} + 7) = \Delta\Phi_i$，所以各次攤銷費用完全相同。

$$\sum c_i = \sum \hat{c}_i + \Psi_0 - \Psi_m = \sum \hat{c}_i - \Phi_m \le \sum \hat{c}_i.$$

雖然 $\Psi_0 = 7$，所有可達終態仍有 $\Psi_m \ge 7 = \Psi_0$。真正需要的是終點不低於起點的條件；「零初態、非負勢能」只是方便使用的充分條件。

A04-E07 常數攤銷不等於常數延遲

queue 連續 ENQUEUE n 筆後，$|S| = n, T$ 空；下一次 DEQUEUE 的 primitive 成本為 $2n + 1$。陣列滿表 (M, M) 時，下一次 APPEND 成本 $M + 1$。兩者都可由合法操作從空結構到達。

總成本界容許工作集中在某一次呼叫，沒有提供每次的 worst-case latency（最壞延遲）界。若要固定每次工作上限，需另一個有最壞界的表示，或把搬移分批、在新舊表示並存時維持正確查詢與更新；分批工作本身也須有完整不變量證明。僅改稱「平均很快」不能補足保證。

A04-E08 中間刪除的成本不在原定理裡

各次刪除分別要移動 $n-1, n-2, \ldots, 0$ 筆，所以搬移合計 $n(n-1)/2$，再加 n 次刪除與 $O(n)$ 建表成本，總計 $\Theta(n^2)$。

只有 $2n$ 次操作，因此這個操作集合不具每次常數攤銷界。正文定理限定 APPEND／DELETE-LAST，且沒有 suffix shift；重新配置的稀疏發生率不能消除每次中間或首端刪除都要做的線性搬移。

A04-E09 四位元溢位的帳目

起態有三個 1，所以 $\Phi_0 = 3$。

結果	c	Φ	$\Delta\Phi$	$\hat{c}$
1110	2	3	0	2
1111	1	4	1	2
0000	4	0	-4	0

實付 $2 + 1 + 4 = 7$，攤銷 $2 + 2 + 0 = 4$；望遠鏡相消得 $7 = 4 + 3 - 0$。每次攤銷至多 2，並不能在非零初態下推出實付至多 $2m$；必須加上初始勢能，這裡得上界 9。溢位只清四個 bit，沒有第五次設為 1。

A04-E10 MULTIPOP 與 MULTIPUSH 的計費單位

實付依序 $1, 1, 1, 2, 1$；餘額依序 $1, 2, 3, 1, 0$。每個前綴不透支，收費總額與實付均為 6。最後一次雖要求移除 9 筆，實際只剩 a，因此只移除一次。

MULTIPUSH(k) 本身就要寫入 k 筆，實付至少 k；從空態只做這一次呼叫即可否定與 k 無關的常數收費。可改為每筆插入收 2，故該呼叫收 $2k$。總成本依輸入元素總數加呼叫數計，不能把可變長參數當成免費輸入。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A04-E11 加一與減一交替的具體反例

準備的翻轉數為 $1,2,1,3,1,2,1$，合計 11，最後是 0111。每組 0111→1000→0111，兩次各翻四個 bit，成本 8。共 $7+8=15$ 次呼叫，實付 $11+4\cdot 8=43$。

一般取 $r=2^{k-1}-1$ 次加一準備，之後做 r 組往返；每組翻轉 $2k$ 次。總呼叫 $m=3r$，實付至少 $2rk=(2/3)mk$，而上界為 mk。因此這族合法序列的成本為 $\Theta(mk)$，不是 $O(m)$。固定 k=4 的數例只能核對算式；要否定獨立於 k 的常數界，必須讓 k 一同增長。

A04-E12 滿佇列拒絕插入後還能正確出列嗎

以 — 表示非成員，不要求真的清掉儲存格。

操作	實體 A[0..2]	h	s	回傳
ENQ A	[A,—,—]	0	1	成功
ENQ B	[A,B,—]	0	2	成功
ENQ C	[A,B,C]	0	3	成功
ENQ D	[A,B,C]	0	3	overflow
DEQ	[—,B,C]	1	2	A
ENQ E	[E,B,C]	1	3	成功
DEQ	[E,—,C]	2	2	B
DEQ	[E,—,—]	0	1	C
DEQ	[—,—,—]	1	0	E

失敗插入必須保持原態；先推進 h 會遺失原首筆 A。此時採拒絕插入規格，不是覆寫最舊資料的 ring buffer。

A04-E13 反轉時為何要先存 nxt

起始 prev=NIL、cur=a→b→c。第 1 輪後 prev=a→NIL、cur=b→c；第 2 輪後 prev=b→a→NIL、cur=c；第 3 輪後 prev=c→b→a→NIL、cur=NIL，故 head=prev。

每輪把原前綴反轉後接到 prev，尚未處理的原後綴由 cur 保留；兩者不重疊且合起來包含原本全部節點。若改接後讀 cur.next 前進，第一輪 a.next 已變成 NIL，cur 立即變 NIL，後綴 b、c 不再可由演算法保存的指標抵達。nxt 必須在覆寫前讀取。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A04-E14 只有一筆資料的 sentinel 串列

插入前 p=z、q=z，寫入依序為 x.prev=z、x.next=z、z.prev=x、z.next=x。完成後由 z.next 與 z.prev 都能到 x，x 的兩邊則都回到 z。

刪除時 p=x.prev=z、q=x.next=z，寫入 z.next=z、z.prev=z，恢復空態。sentinel 是永久的結構錨點，不是資料元素；允許回收 z 會使空態表示、搜尋終點與後續操作失去有效物件。刪除成本的前提包括 x 是有效資料成員且 $x \neq z$。

A04-E15 回收槽位之後，舊 handle 還代表原物件嗎

配置 1 後 free=4，D 接入後 head=5，live 為 5(A)→2(B)→3(C)→1(D)→0。刪 B 時改 next[5]=3、prev[3]=5；FREE(2) 改 next[2]=4、free=2。最後 live 為 5→3→1→0，free 為 2→4→0，兩者互斥且合起來包含五個槽。

再次配置取得 2，free 變 4；該槽暫由呼叫者持有，尚未接入 live。舊 handle 雖數值相同、仍在範圍內，卻不再代表 B。呼叫端必須遵守失效規則；若需要偵測陳舊 handle，可另設世代編號並驗證，但這是額外機制，不能假設目前的純索引已提供。

A04-E16 繞回、刪除與同鍵更新

呼叫	探測槽	結果
INSERT(4,a)	4	寫槽 4
INSERT(9,b)	4,0	寫槽 0
INSERT(14,c)	4,0,1	寫槽 1
DELETE(9)	4,0	槽 0 改 DELETED
INSERT(14,c′)	4,0,1	更新槽 1
INSERT(19,d)	4,0,1,2	在槽 0 放 (19,d)
FIND(24)	4,0,1,2	槽 2 是 EMPTY，回不存在

最後槽 0～4 為 [(19,d),(14,c′),EMPTY,EMPTY,(4,a)]，n=3。插入 14 在槽 0 已遇 tombstone，但同 key 在槽 1；立即寫入會使兩槽都存 14。新鍵 19 則先到槽 2 確認不存在，才重用槽 0。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A04-E17 Affine family：真的枚舉一次

先比較 r=(a+b) mod5、s=(4a+b) mod5，再各取 mod3。相異 r、s 只有相差 3 時才會合併，亦即有序對 (0,3)、(3,0)、(1,4)、(4,1)。

a	碰撞的 b
1	0,4
2	無
3	無
4	0,4

20 種參數中 4 種碰撞，機率 1/5≤1/3，滿足「至多」版本。本題只驗證一對鍵；一般 universal 必須對所有固定相異鍵成立，不能用這份枚舉取代一般證明。

A04-E18 步長不互質的代價

step=2 的不同位置為 1,3,5,7，接著回 1；週期 8/gcd(8,2)=4。槽 0 不可達，因此不能插入；即使執行 8 次也只是重複這四格。

step=3 的順序為 1,4,7,2,5,0,3,6，週期 8，遍訪全表。三角偏移為 0,1,3,6,2,7,5,4，也是排列。$i(i+1)$ 為偶數，要先在整數中除 2，再取 mod8；2 在模 8 中沒有乘法逆元素，不能寫成乘上 2 的模逆。

A04-E19 理想模型的精確期望與上界

尾機率為 1、2/4=1/2、(2/4)(1/3)=1/6；再大為 0。所以 E[X]=1+1/2+1/6=5/3，上界 1/(1−1/2)=2，並不取等號。

第一個占用槽已被探測後，剩三槽只有一槽占用，條件機率是 1/3，不是 1/2。這是不放回排列的抽樣；將乘積上界換成 α^2 可以做估計，不能把它當成精確機率。

A04-E20 FKS 空間與成員資格

桶大小為 [0,1,2,2,1,0]，S=0+1+4+4+1+0=10。只有兩個雙筆桶各有一對碰撞，Y=2，故 n+2Y=6+4=10<24。

26 到桶 2、次層槽 2，讀到 26，成功。14 走同路卻讀到 26，不存在。5 到空桶 5，不存在且不算次層。10 到單筆桶 4，讀到 4，不存在。

省略完整 key 比較會把 14、10 誤判存在。平方和合格與桶內無碰撞各須獨立檢查：前者控制空間，後者保證已存鍵不互撞，兩者都不省略 query 的相等判斷。

A04-E21 刪空之後，為何不是一次就查完

n=0、d=5，所以 α=0、β=1。FIND(7) 的 home 是 2，探測 2,3,4,0,1；因沒有 EMPTY，走完五次才回不存在。

沒有反證：定理假設無 tombstone 且採理想探測排列，此例兩項都不符合。即使換成任何全表排列，五個 DELETED 仍迫使走完五槽；關鍵是不能把活躍負載 α 當成停止機率。若重建為全 EMPTY，才可在第一次探測停下，而重建成本須另計。

A04-E22 合數 modulus 破壞哪一步？

h_a(0)=0，h_a(2) 依 a=0,1,2,3 分別是 0,2,0,2。兩種 a 碰撞，機率 2/4=1/2>1/4，故這個 family 在該 universe 上不符合本章 universal 定義。

差值 2 在模 4 下非零卻不可逆，方程 2a=0 mod4 有兩個解，而不是唯一解。失效的是「非零差值有逆元素」這一步，不是線性期望。結論針對這個構造；並非聲稱任何合數大小的 hash table 都不能 universal，前面的 affine family 就允許任意 m。

A04-E23 父子關係都對，整棵樹卻錯了

25 位於 20 的左子樹，必須 <20；同時在 10 的右子樹，必須 >10，因此允許區間是 (10,20)，25 不在其中。FIND(25) 比20後往右，到30後往左，抵達 NIL，回不存在。錯誤在資料表示已破壞 BST 不變量，不能因此判定正確 BST 上的搜尋程序有誤。

A04-E24 相同樹，不同查詢分布

五個成功查詢的比較數為1,2,2,3,3，平均11/5。失敗1,4,6,9,10的比較數為2,3,3,2,2，總和12，因此混合平均23/10。若只查9，路徑5→8→NIL，平均2。三個值不同是機率模型不同，不能只給樹形就宣稱唯一的「平均搜尋成本」。

A04-E25 後繼的兩條路徑

5 有右子樹，從8沿left到7，答案7。3 沒有右子樹，先到父2；3是2的右孩子，繼續到父5，而2是5的左孩子，答案5。8 沒有右子樹，向上到5後仍屬右孩子，再到NIL，沒有後繼。祖先可能提供最小的較大鍵，所以無右子樹時還需檢查向上路徑。

A04-E26 直接後繼為什麼不能照抄深處分支

因 z.right 就是 y，指定後成為 y.right=y，自環會使走訪不終止，也讓50失去原連結。正確做法跳過深處分支，改成root=y、y.parent=NIL、y.left=10、10.parent=y，保留y.right=50、50.parent=y。原物件20離開，其他三個物件保留。

A04-E27 後繼帶著右孩子的根刪除

後繼為25，右孩子是27。更新如下：

步驟	寫入
先接替後繼原位置	30.left=27；27.parent=30
後繼接原右子樹	25.right=40；40.parent=25
後繼接替根	root=25；25.parent=NIL
接原左子樹	25.left=10；10.parent=25

其餘連結維持，最後中序為10,25,27,30,40,50。移接法令原20物件失效，原25物件仍有效但位置變了；複製法則保留原20物件、把其key與value改為25的記錄，並令原25物件失效。不得只複製key而留下20的value。

A04-E28 六種排列的期望高度

213、231得到高度1，其餘123、132、312、321高度2。所以$E[X_3]=(2\cdot 1+4\cdot 2)/6=5/3$，$E[2^{X_3}]=(2\cdot 2+4\cdot 4)/6=10/3$，而$2^{(5/3)}$約3.175<10/3。若五種樹形各以1/5抽取，平均變成$(1+4\cdot 2)/5=9/5$；這與均勻排列的5/3不同。

A04-E29 固定秩與平均秩的期望搜尋成本

$H_3=1+1/2+1/3=11/6$，所以$E[D_3]=2H_3-2=5/3$，搜尋比較為8/3。$H_5=137/60$；總深度期望$2\cdot 6\cdot 137/60-20=37/5$，因此均勻成功查詢的平均比較數期望為$(37/5)/5+1=62/25$。固定秩3與隨機抽一個已存key並非同一實驗。

A04-E30 指數高度證明的基例與不等號

$Y_0=0$是人為輔助定義，只在$n\geq 1$時令$Y_n=2\text{^}X_n$；單節點另驗$Y_1=1$，遞迴從$n\geq 2$開始。期望不能穿過max；應條件在root秩，套$\max(u,v)\leq u+v$，得$a_n\leq (4/n)\Sigma_{j=0}^{n-1}a_j$。歸納假設與立方和給$a_n\leq (4/n)[n(n-1)/2]^2=n(n-1)^2\leq n^3$。因2^x凸，Jensen給$2\text{^}E[X_n]\leq E[2\text{^}X_n]=a_n\leq n^3$；取$\log_2$得到$E[X_n]\leq 3\log_2 n$。結論只約束期望，不排除個別排列生成高度$n-1$的鏈。

A04-E31 • 黑高與路徑黑數

原色：bh(20)=1、bh(10)=1、bh(NIL)=0；完整路徑含20與NIL，黑數為2。改色後：bh(20)=2、bh(10)=1、bh(NIL)=0；完整路徑含20、10或30、NIL，黑數為3。黑高排除起點，不能把完整路徑黑數直接填入 bh。

A04-E32 • 左旋的外父與空子樹

依序10.right=15；15.parent=10；20.parent=30；30.left=20；20.left=10；10.parent=20。根情況改20.parent=NIL、T.root=20。β=NIL時省略第二項，不能覆蓋共用NIL.parent的修復上下文。y原父就是x；應改的是x舊父p的同一child slot指向y。旋轉前後中序均為10,15,20（若算上外父則接30），物件身分不變。

A04-E33 • 非根 I1 的黑數

改色前bh(g)=b，含g的黑數=1+b；改色後bh(g)=b+1，含g的黑數仍=1+b。因此非根g以上的各條路黑數保持，不是同增一黑。只可能新生g與其父的紅紅。若g是root，最後把g塗黑，bh(g)仍b+1（不含自己），含root完整黑數變b+2；所有路一起增加，故仍相等。

A04-E34 • I2 不能提前返回

插入後root35B，35.left=15R、15.right=25R，z=25。I2先z=15，左旋15後35.left=25R、25.left=15R，parent分別為NIL、35、25，紅紅仍在15/25。I3把25黑、35紅，右旋35後root25B、25.left=15R、25.right=35R，兩child.parent=25，root.parent=NIL，z仍15。鏡射順序則先右旋35（z=35），再將25黑、15紅並左旋15；同得25B及兩紅child。每個最終root–NIL路有2黑，過程至多2次基本旋轉。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A04-E35 • 高度證明的計數邊界

NIL基例為0=2⁰−1。內部節點的兩child黑高至少bh(x)−1，歸納得到

$$n_x \geq 2(2^{bh(x)-1}-1)+1 = 2^{bh(x)}-1.$$

最長路徑含B黑、R紅，兩端黑且不連續紅，所以R≤B−1；H=B+R−1≤2(B−1)=2bh(root)。結合節點下界得$n \geq 2^{H/2}-1$，取log即得結論。空樹h=−1、H=0；單節點h=0、H=1。n=0時log(n+1)=0，不能涵蓋必要常數測試；可寫O(1+log(n+1))或O(log(n+2))。

A04-E36 • 不同修復時機的比較

CLRS bottom-up中，只有I1能反覆往上走，且不旋轉；遇I2/I3最多做2次基本旋轉後即結束。因此總旋轉數有更強的常數上界2，O(log(n+2))總時間來自搜尋與可能逐層重上色。top-down在搜尋途中先分裂4-node，CLRS在置入紅葉後處理紅紅，入口不變量與變色時機不同，可能得不同合法樹形。「一次雙旋」算2次基本旋轉，不能和一次單旋混為同一計數。

A04-E37 • 原色與移動後顏色

z=40B，y=45R，x=NIL。先改50.left=NIL、NIL.parent=50；45.right=60、60.parent=45；root=45、45.parent=NIL；45.left=20、20.parent=45；最後45.color=B。y原紅，舊位置沒有少黑，不做fixup。移動後的黑色是承接z原位置的貢獻，不能當成舊位置缺黑的證據。其餘record身分與key/value不變，只有40離開；操作結束才可把NIL.parent重設NIL。

A04-E38 • 最後一點與紅 child 吸收

單節點：移接後root=x=NIL，所以迴圈條件x!=root為假；把x.color設B不改sentinel色，空樹合法，root.parent=NIL。第二例移接後root=x=15R，也不進迴圈；最後15塗黑，吸收被移除25的額外黑。實際color始終只有R/B；多算的一黑由cursor x與證明中的路徑計數表達。不能把NIL塗紅，也不能為空樹執行普通節點刪除。

A04-E39 • D1 後為何還要 D2

刪後x=NIL=25.left，p=25B，w=45R。D1將45黑、25紅，左旋25，root變45；x仍在25.left，新w=35B。35兩child皆NIL黑，D2把35紅並回傳p=25R。外層因x紅退出，最後25黑。結果45B：左25B（右35R），右55B（右65R）。只旋轉一次；D1僅轉換兄弟顏色，extra由25最後塗黑吸收。

A04-E40 • D3→D4 的逐路黑數

D3前：左路實際2、extra1、有效3；右路實際3、extra0、有效3。D3把35黑、45紅並右旋45，root仍25，右為35B（右45R）；左缺黑位置未動，所以左路仍2+1=3，兩種右路25B→35B→NIL與25B→35B→45R→NIL皆3。D4將35繼承25的黑、25黑、45黑，再左旋25。最終35B、左25B、右45B；所有根至NIL路實際3、extra0。D3未消除extra，只有D4完成修復，共兩次基本旋轉。

A04-E41 • 右側缺黑與直接後繼

z=35B，y=45B是直接右child，x=y.right=NIL，因此先設NIL.parent=45；45承接root後，x是45.right。w=15B在左；near=w.right=25R，far=w.left=NIL黑。鏡射D3把25黑、15紅並左旋15，重新取w=25B；鏡射D4使25承接45原黑，45與far15黑，再右旋45。最終root25B、左右15B與45B。若仍用left當near，便會檢查錯姪節點；若parent留在已刪35，則會從失效物件找兄弟。

A04-E42 • 為何刪除最多三次旋轉

D2不旋轉，將唯一extra從x交到父，每次嚴格上移一層；父紅則退出並吸收，父為root則全樹同去extra，否則繼續。D1把原p變紅，後續若D2便由紅p吸收；若不D2則直接D4，或D3接D4結束。因此D1不可能出現後還再往上遇另一個D1。D3必立即接D4；D4消除extra並返回root，兩者亦各至多一次。故基本旋轉總數≤3，但搜尋、找後繼及D2逐層上移仍需O(log(n+2))。

A04-E43 • 局部秩與全域秩

select 在 20 比較 k=7、r=4，往右改 k=3；在 40 比較 3 與 5，往左仍為 3；在 30 的 r=3，回傳 30。rank(27) 起始 1，27→25 因右 child 加 1 得 2，25→30、30→40 不加，40→20 加 size(10)+1=4，得 6。空樹無合法 select；重複鍵須以 (key,identity) 定全序，rank 回答的是物件位置，不能只給值便假定唯一位置。

A04-E44 • 後繼搬移後的摘要

移接前保存 a=30。完成全部連結後，27 填 30.left，新根 25 連左 10、右 40。沿最終 parent 鏈 30→40→25 重算，依次為 1+1+1=3、1+3+1=5、1+3+5=9。舊根的 parent=NIL，從那裡出發完全不會修到受影響的三點。先算 25 也錯，因為它會讀取尚未更新的 40.size=6。

A04-E45 • 局部公式不等於只改兩格

旋轉保留整棵局部子樹的物件集合，因此新 y 的 size/max 等於舊 x 的摘要，外部祖先看到的摘要不變。完成指標改接後先 pull(x)，再 pull(y)，因新 y 依賴新 x。height 雖由兩個 child 的 height 算出，局部樹高卻可能變化，須向祖先傳播。插入新最小鍵時所有舊物件的全域 rank 都加 1；若每格明存便需 Θ(n) 次寫入。

A04-E46 • max 大一點也不安全

在 root[12,17] 因 left.max=27≥22 往左；在 [5,7] 因 left[2,4].max=4<22 往空 right，錯回 NIL。右子樹 [20,24] 其實重疊。正確 max=7 會讓第一步往右。往左的證明需要左子樹內確有一個 high 等於 max 的見證 j；任意上界 27 沒有相應 j，無法推出若左無解則右亦無解。

A04-E47 • 相接端點與欄位更新

閉區間在 5 重疊，overlap 為兩個 ≤，左側測試 ≥。半開沒有交集，overlap 改兩個 <，左側測試 >；[5,5) 為空，必須依契約拒絕或另行處理。low 是排序鍵，修改時須刪除再插入；high 不影響鍵序，但驗證非空後須從該節點向上 pull max，允許摘要下降。

A04-E48 • 列全部答案的成本

除了 k 次成功查詢，還有一次失敗查詢確認已列完；k=0 時這次工作仍存在。以原樹規模 n 作統一上界，查詢、刪除和插回共 O((k+1)log(n+2))，另需 O(k) 儲存結果，且過程暫時修改集合。它沒有證明 O(log n+k)；若要更緊的輸出敏感界，須換演算法或另外證明訪問節點數。

A04-E49 • 兩種容量參數與分裂時機

m=3 的下限 ⌈3/2⌉−1=1、上限 2；m=4 為 1…3；t=2 也是 1…3。對 [2,8] 先提升一鍵後，只剩一鍵可分配到兩側，必有一側零鍵而違反下限。溢位後先有 [2,5,8] 再提升 5，才留下兩個合法的一鍵節點。對應公式是 m=2t，不是任意 m 都能直接代換。

A04-E50 • 十億鍵的高度與頁讀取

密集界 $n \le 200^{h+1} - 1$：h=2 上限 7,999,999，不夠；h=3 上限 1,599,999,999，足夠。稀疏界 $n \ge 2 \cdot 100^{h} - 1$：h=4 下限 199,999,999 仍可，h=5 下限 19,999,999,999 太多，所以 h∈{3,4}。路徑節點數 h+1；根不讀，因此讀 h=3 或4頁。若連根都未快取，變成4或5頁。

A04-E51 • 內節點分裂的全部子指標

提升中位鍵20，父變[20,50]，其三個子節點依序為[10]、[30]、[70]。[10]連[5]與[15]；[30]連[25]與[35]；[70]仍連[60]與[80]。20不留在子節點。中序5,10,15,20,25,30,35,50,60,70,80與原本相同，全部葉深度仍2，每個一鍵內節點皆有兩個子指標。

A04-E52 • 借鍵再合併的連續刪除

先將根20放入左內節點[10,20]，右兄弟最小鍵30上移為根，右內節點剩[40]。[25]需搬成左內節點的最後子樹，因20<25<30。接著將[5]、分隔鍵10與[15]合併，再刪5，得[10,15]。最後根[30]；左內節點[20]連[10,15]、[25]；右內節點[40]連[35]、[45]。若[25]留右側，便在根30的右子樹中出現25<30，違反全域範圍。

A04-E53 • 前驅與後繼的資料一致性

A用前驅10，先從左子樹刪10再以其整筆記錄替換根，結果根[10]、葉[5]與[30]。B左只有1鍵、右有2鍵，所以用後繼30，結果根[30]、葉[5]與[40]。替換只改key卻保留record20的資料，搜尋10或30會找到正確鍵卻回傳錯資料；必須保存整筆資料或穩定record pointer，避免刪除原位置後再讀失效記憶體。

A04-E54 • 失敗刪除也要處理根

原本選child1；兩葉皆最小，向左合併，目標索引改為0。形成[5,20,30]後查無99，回NOT_FOUND。舊根已零鍵且一個child，仍須縮高，最後葉根[5,20,30]，集合不變。提早退出會留下不合法的零鍵內根。失敗只保證未刪元素；若要求樹形也不動，應先做存在性查詢。

A04-E55 • 根為何可以縮高

先合併成[5,20,30]，刪20為[5,30]，以它取代空內根，h由1降0。再刪5得葉根[30]，h=0；刪30後保留零鍵葉根，依約定h=0。根縮高使所有葉路徑同減一層；一般內節點縮掉一層只改到部分後代，其他葉深度不變，破壞等深度。此外正確的下行刪除不會讓非根內節點合法地降到零鍵。

A04-E56 • 頁內二分搜尋能改善哪些成本

三種操作的頁存取皆O(h+1)，因每層只處理常數個頁。搜尋頁內二分需O(log t)，總O((h+1)log t)比較；線性掃描則O(t(h+1))。插入、刪除即使用二分找到位置，陣列鍵與子指標仍可能搬移Θ(t)個，故CPU上界仍O(t(h+1))。固定單頁容量、快取與寫入模型是分析前提；頁存取數也不是實測延遲。

A04-E57 • 環串列與合併契約

先存 al=a.left、bl=b.left，再設 al.right=b、b.left=al、bl.right=a、a.left=bl。這四個接點把兩條環路接成一環，其他節點連線不變，故 O(1)。結果與舊入口若同時作為獨立堆積操作，會共享並修改同一組節點，造成 n、min 與串列互相矛盾。必須消耗舊入口；空環另取另一入口，不能解參考 NIL。

A04-E58 • 完整整理與孩子提升

第一次 link 為 9→4、12→7、7→4、15→6，箭頭為孩子指向父。終態根 4(9,7(12))、6(15)、10。第二次移除 4 並提升 9、7，根順序 10(degree0)、6(degree1)、9(degree0)、7(degree1)。處理 9 時先 10→9，再 9→6，得到 degree2 的根6；最後7留在degree1。終態根6有15、9(10)，根7有12；min=6，n=6。父鍵≤子鍵、根 degree 為2與1，總大小4+2=6。

A04-E59 • 降低鍵值不等於單獨搬走一個值

15→14沒有破壞14≥12，全部指標、degree與mark不變，min=1。15→11時切整棵11(20)，所以11.parent=NIL且mark=false，20.parent仍為該節點11。12.degree由1降0，12為非根且第一次失去child，故mark=true；根1不標記。根串列為1與11，11.degree仍1，min仍1。

A04-E60 • 祖父為根與非根的兩個結局

g=2為根時，先切x，再切p並清p.mark；到g直接停。根{2,1,8}，p仍帶15，所有根未標記；(r,m):(1,1)→(3,0)，未縮放勢能3→3，min=1。若q=0為根、g=2為未標記非根，則兩次切割後標記g而停；根{0,1,8}，q仍接g、p仍帶15，只有g標記；(1,1)→(3,1)，勢能3→5，min=0。

A04-E61 • 攤銷證明中的常數不能憑空抵銷

c≥1時新增c根；除第一次外其餘c−1次切掉已標記節點，結尾至多加一個mark，故Δm≤2−c，ΔΦ≤K(4−c)。取固定K≥a，ĉ≤a(c+1)+K(4−c)≤a+4K。O(c)隱藏係數可能大於1，未選尺度就不能讓它和−c抵銷。同一K也須涵蓋取出最小值的根處理常數；只需取所有相關固定係數的最大值，不隨操作次數或c改變。

A04-E62 • 不用樹高證明degree界

接入時目前仍存的較早i−1個孩子都已在父下，故父degree≥i−1；同degree才能link，所以新孩子當時degree也≥i−1。它若再少兩個孩子便被切走，仍留下只可能少至多一個，故≥i−2。子樹大小歸納可套每個孩子，得到d≥2時size≥2+$\sum_{i=2}^{d} F_i$=F_{d+2}；d=0、1分別至少1、2。再由F_{d+2}≥φ^d與n≥size得$d \le \log_\varphi n$。這限制直接孩子數，不限制最長父子路徑，不能推成最壞遞減鍵只需O(log n)。

A04-E63 • 重複最小值與刪除身分

若min原指u，嚴格較小才更新min的實作不會改成x，接著EXTRACT-MIN就刪u。平常比較鍵設(1,key)，本次只有x設(0,任意值)，使x小於所有一般鍵並成為唯一最小；經DECREASE-KEY後再EXTRACT-MIN，最後還原取出物件的原鍵。兩步共享同一勢能，中間項相消，總攤銷O(log(n+1))，搜尋x的成本仍須另算。

A04-E64 • 延遲工作的真實代價

插入後n個單節點根，Φ=Kn；取出時必須處理其餘n−1根，故實際Θ(n)。整理後根數至多O(log(n+1))，清mark不增加勢能，因此釋放先前插入累積的勢能支付掃根與link。每次插入的攤銷O(1)已把這份成本存入，序列總成本O(n+log(n+1))。一般初態由$\sum_i c_i = \sum_i \hat{c}_i + \Phi_0 - \Phi_{\text{final}} \le \sum_i \hat{c}_i + \Phi_0$，不能省去可能很大的初始勢能。

A04-E65 • 奇數位元寬度與局部座標

頂層 m=5、H=8、L=4：21→(5,1)，30→(7,2)，分別還原為5×4+1=21、7×4+2=30。21 的 low=1 進入 u=4 的 C_5；此層 m=2、H=L=2，1→(0,1)，還原0×2+1=1。再進入 u=2 的局部 C_0便是基例，不再建立H/L。summary 的 universe 是8，不是4。

A04-E66 • 重建集合與三種後繼

min=2、max=30；C_0={3}、C_1={3}、C_4={2}、C_5={1}、C_7={2}，summary={0,1,4,5,7}。查3時i=0,j=3，j不小於C_0.max，所以走summary得下一組1，取C_1.min=3，還原7。查20時i=5,j=0<C_5.max=1，所以走C_5得1，還原21。查30碰到x≥max直接回NIL，不能回30本身。

A04-E67 • 前驅不是後繼的完全鏡像

PREDECESSOR(3)=2：C_0只有3，沒有更小局部鍵；summary也沒有小於0的組號，但頂層min=2且2<3，因此回2。PREDECESSOR(2)=NIL，因x≤min。PREDECESSOR(31)=30，因x>max且31仍在合法universe。漏掉獨立min的版本會把第一題誤判NIL。

A04-E68 • 換出舊 min 與首次非空的組

插入1先與min2交換；舊2進C_0，局部集合{3}→{2,3}，summary仍{0,1,4,5,7}，min=1,max=30。插入10分解為i=2,j=2；C_2原空，所以先在summary插入2，再把C_2設min=max=2。結果summary={0,1,2,4,5,7}。這次C_2的初始化是O(1)，昂貴路徑在summary；前次則是非空C_0的遞迴。

A04-E69 • 提升最小值與縮成單一元素

兩例均讀summary.min=0與C_0.min=3，把x及頂層min改成3，再從C_0刪low(3)=3。C_0原為單一元素，立即清空，接著刪summary的0。S的summary成{1,4,5,7}，x=3≠max30，所以max仍30。T的summary變空，x=3等於舊max，故max=min=3。新min不再留在任何cluster。

A04-E70 • 兩次遞迴呼叫的真正成本

只有cluster刪空才刪summary；它原本只能是單一元素，因此第一次呼叫O(1)。若原本至少兩鍵，刪除後仍非空，不刪summary。所以 $R(m) \le R(\lceil m/2 \rceil) + c$，R(1)=O(1)。m≥2時ceil(m/2)≤2m/3，故深度O(log m)，總成本 $O(1 + \log\log u)$。這是最壞分析，不是兩個昂貴呼叫的攤銷抵消。

A04-E71 • 完整配置的空間歸納

選B≥c、A≥B+P(2)，基例P(2)≤A−B成立。子項代入得 $P(u) \le A[H(L-1) + H - 1] - B(H+1) + cH \le A(u-1) - B$，故O(u)。只沿cluster分支展開的底層已需Ω(u)，所以Θ(u)。逐節點配置與清空要Θ(u)時間；q次操作總上界為 $O(u + q(1 + \log\log u))$，不能從空樹假裝免費開始。

A04-E72 • 違反前提會如何破壞資料

DELETE-PRESENT看到min=max=1，會把兩欄清NIL，誤刪合法鍵1。INSERT-ABSENT(1)會將1的low放進C_0，同時頂層min仍為1，造成min也出現在cluster。公開介面先檢查整數範圍$0 \le x < u$，再MEMBER；存在才刪，不存在才插，其他情況回明確狀態且不更動集合。查找與更新相加仍為最壞O(1+log log u)。

參考資料與署名

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (2009), Chapters 10–14, 17–20. MIT 6.046J/18.401J (2004), L09: Balanced Search Trees, Piotr Indyk. MIT 6.046J/18.401J (2005), L08: Hashing II; L09: Randomly Built BSTs; L11: Augmenting Data Structures; L13: Amortized Analysis. Charles E. Leiserson and Erik D. Demaine. Lin Cing，個人手寫資料結構筆記：佇列、搜尋樹、紅黑樹、多路搜尋樹與堆積。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study